

Data Mining



Classification Decision Tree

Why Decision Trees?

- Interpretable model (if–then rules)
- Handles categorical + numeric
- No scaling needed (contrast with KNN/SVM)
- Used in Random Forest / XGBoost

An Example Dataset

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

A Decision Tree for “*buys_computer*”

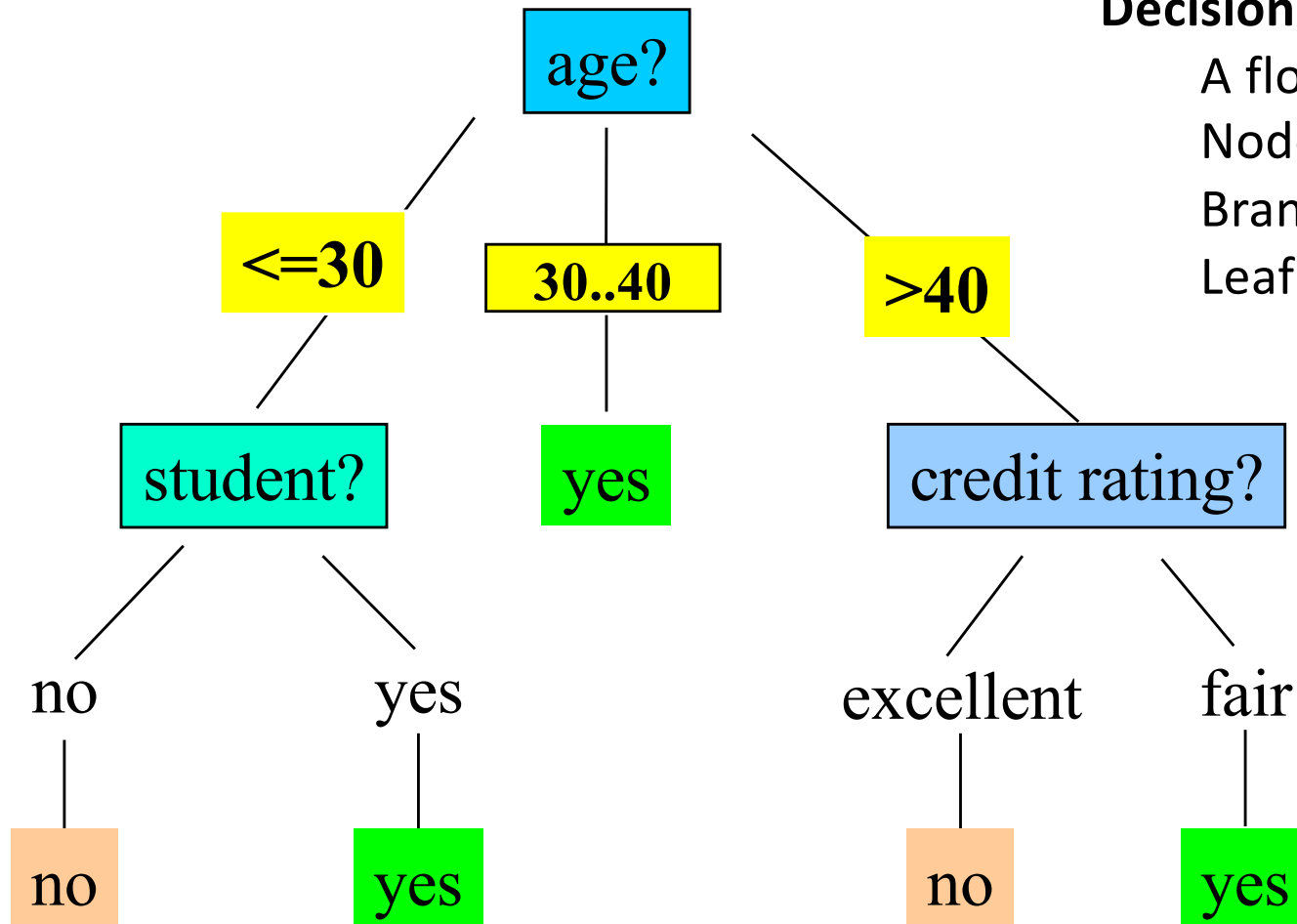
Decision tree

A flow-chart-like tree structure

Node : feature test

Branch : condition

Leaf : prediction



age ≤ 30 → student = yes → predict = yes

Classification by Decision Tree Induction

- Decision tree generation consists of two phases
 - Tree construction
 - At start, all the training examples are at the root
 - Partition examples recursively based on selected features
 - Tree pruning
 - Identify and remove branches that reflect noise or outliers
- Use of decision tree: Classifying an unknown sample
 - Test the feature values of the Query data against the decision tree

Algorithm for Decision Tree Induction

```
function build_tree(data):
```

```
    if stopping_condition:  
        return leaf
```

```
    best_feature = argmax InformationGain(feature)
```

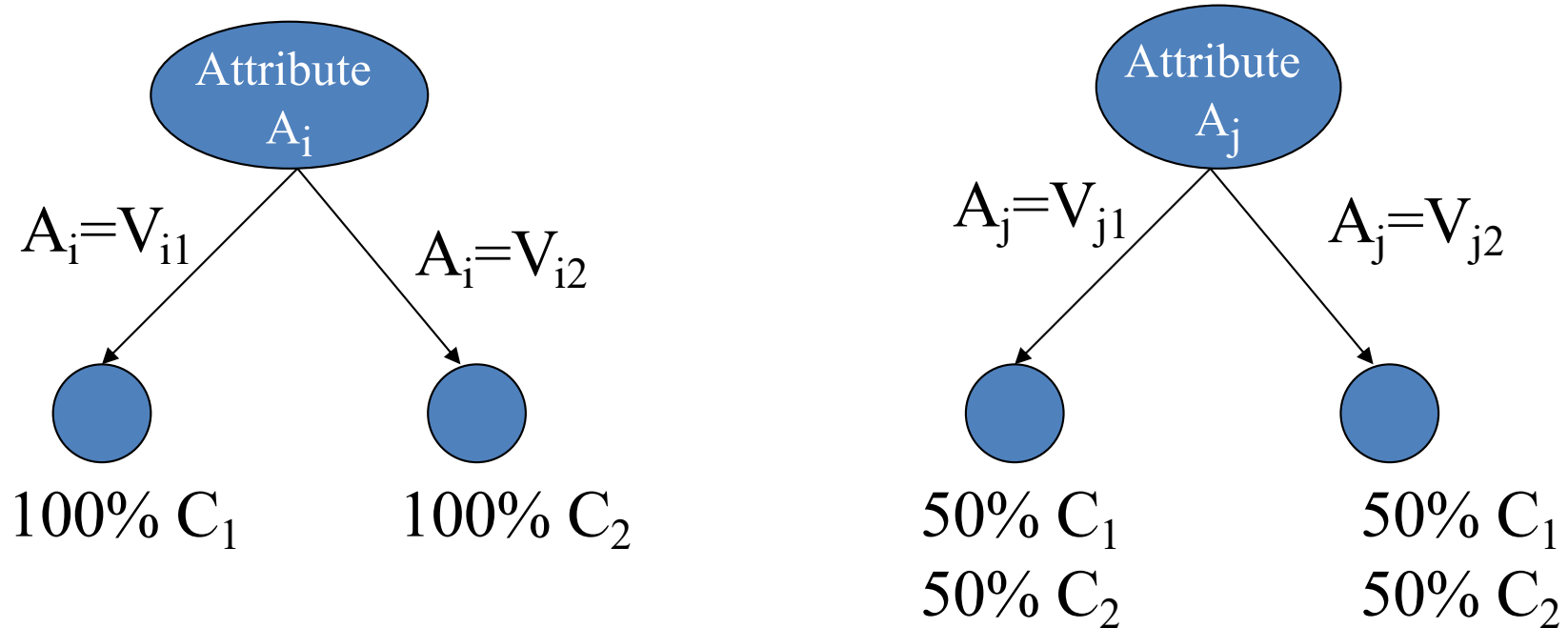
```
    for each value v:  
        subset = data where feature = v  
        child = build_tree(subset)
```

```
    return node
```

Attribute Selection

- Which split makes the data MOST PURE?
- **Information gain**: choose attribute with largest expected information gain, i.e., choose attribute that will result in the smallest expected size of the sub-tree rooted at its children.
 - **Occam's Razor**: The simplest explanation that is consistent with all the observations is the best → smallest decision tree that correctly classifies all of the training examples is the best

Attribute Selection



Which attribute is better at splitting up the data ?

Good split $\rightarrow$ child nodes are pure

Attribute Selection Measures

Measure	Used in	Idea
Entropy/IG	ID3	Uncertainty reduction
Gain Ratio	C4.5	Penalize many splits
Gini	CART	Impurity

Entropy + Information Gain

X = College Major

Y = Likes “Gladiator”

Definition of Information Gain:

$$IG(Y|X) = H(Y) - H(Y|X)$$

X	Y
Math	Yes
History	No
CS	Yes
Math	No
Math	No
CS	Yes
History	No
Math	Yes

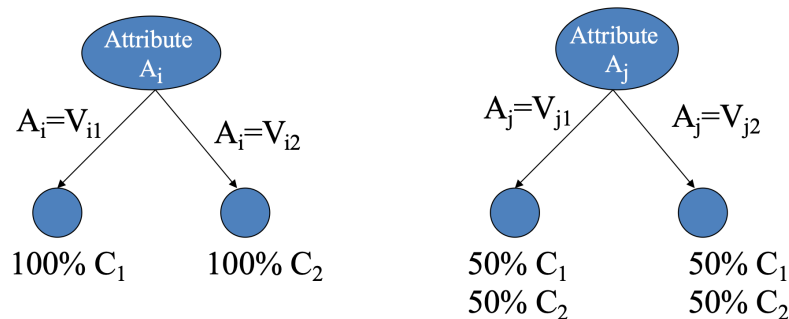
- H : the entropy, measuring the uncertainty in labels
- H(Y): the uncertainty of class label Y
- H(Y|X) the uncertainty of label Y after observing X
- The **more** the **reduction** in uncertainty, the **higher** the **Information Gain**

Entropy

- **Entropy:** $H(S) = - \sum_{k=1}^K p_k \log_2 p_k$ p_k : the probability of a class
 - **Uncertainty in Class Labels**
 - **Pure node: entropy = 0**
 - **Mixed node: high entropy**

Class distribution	Entropy
C1: 100%, C2: 0%	0
C1: 10%, C2: 90%	0.469
C1: 30%, C2: 70%	0.881
C1: 50%, C2: 50%	1

Which feature is better?



Entropy - General Case

A histogram of the frequency distribution of values of X would be flat

A histogram of the frequency distribution of values of X would have many lows and one or two highs

$H(X)$ = The entropy of X

- “High Entropy” means X is from a uniform (boring) distribution
- “Low Entropy” means X is from varied (peaks and valleys) distribution

Entropy for Attribute Selection

- Entropy of a class C_k :

$$P(C_k) \cdot (-\log_2 P(C_k))$$

- Entropy of one node containing K classes is:

$$H(\text{node with K classes}) = \sum_{k=1}^K P(C_k) \cdot (-\log_2 P(C_k))$$

- For an attribute A_i , having J different values, entropy of all child nodes resulting from selecting this attribute is:

$$H(J \text{ child nodes}) = \sum_{j=1}^J P(A_i = V_{ij}) \cdot \sum_{k=1}^K P(C_k | A_i = V_{ij}) (-\log_2 P(C_k | A_i = V_{ij}))$$

Maximize information gain

Information Gain = $H(\text{parent node}) - H(J \text{ child nodes})$

$$= \sum_{k=1}^K P(C_k) \cdot (-\log_2 P(C_k)) - \left(\sum_{j=1}^J P(A_i = V_{ij}) \cdot \sum_{k=1}^K P(C_k | A_i = V_{ij}) (-\log_2 P(C_k | A_i = V_{ij})) \right)$$

- Use Information gain for attribute selection:
 - Information gain measures how much uncertainty is reduced after a split.
 - A good split produces **purier child nodes** (lower entropy).
 - The greater the reduction in entropy, the better the split.
- How to choose the best attribute:
 - Compute information gain for each candidate attribute
 - Select the attribute with the **highest information gain**

Information Gain for Attribute Selection

Suppose you are trying to predict whether someone is going to live past 80 years. From historical data you might find...

- $IG(\text{LongLife} \mid \text{HairColor}) = 0.01$
- $IG(\text{LongLife} \mid \text{Smoker}) = 0.2$
- $IG(\text{LongLife} \mid \text{Gender}) = 0.25$
- $IG(\text{LongLife} \mid \text{LastDigitOfSSN}) = 0.00001$

Which attribute best predicts the longevity?

Example (1)

Given the following data, which attribute should be selected to divide the data and build the first level of the decision tree?

Object	Color	Shape	Size	Class
D1	Red	Square	Big	+
D2	Blue	Square	Big	+
D3	Red	Round	Small	--
D4	Green	Square	Small	--
D5	Red	Round	Big	+
D6	Green	Square	Big	--

Example (2)

Attribute = Color

Split data:

$$P(\text{red}) = \frac{3}{6}, \quad P(\text{blue}) = \frac{1}{6}, \quad P(\text{green}) = \frac{2}{6}$$

Color = red

$$P(+ \mid \text{red}) = \frac{2}{3}, \quad P(- \mid \text{red}) = \frac{1}{3} \quad H(\text{Class} \mid \text{red}) = - \left(\frac{2}{3} \log_2 \frac{2}{3} + \frac{1}{3} \log_2 \frac{1}{3} \right) = 0.9183$$

Color = blue

$$H(\text{Class} \mid \text{blue}) = 0$$

Color = green $H(\text{Class} \mid \text{green}) = 0$

Information Gain:

$$IG(\text{Color}) = 1 - \left[\frac{3}{6}(0.9183) + \frac{1}{6}(0) + \frac{2}{6}(0) \right] = 0.5408$$

Example (3)

Entropy of Parent: $P(+) = \frac{3}{6} = \frac{1}{2}, \quad P(-) = \frac{3}{6} = \frac{1}{2}$

$$H(\text{Class}) = - \left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2} \right) = 1$$

Attribute = Size

Split data: $P(\text{big}) = \frac{4}{6}, \quad P(\text{small}) = \frac{2}{6}$

Node: Size = big (4 samples)

$$P(+|big) = \frac{3}{4}, \quad P(-|big) = \frac{1}{4}$$
$$H(\text{Class} | \text{big}) = - \left(\frac{3}{4} \log_2 \frac{3}{4} + \frac{1}{4} \log_2 \frac{1}{4} \right) = 0.8113$$

Node: Size = small (2 samples)

$$P(+|small) = 0, \quad P(-|small) = 1$$
$$H(\text{Class} | \text{small}) = 0$$

 Information Gain

$$IG(\text{Size}) = H(\text{Class}) - H(\text{Class} | \text{Size}) = 1 - 0.5408 = 0.4592$$

A small dataset: Miles Per Gallon

mpg	cylinders	displacement	horsepower	weight	acceleration	modelyear	maker
good	4	low	low	low	high	75to78	asia
bad	6	medium	medium	medium	medium	70to74	america
bad	4	medium	medium	medium	low	75to78	europa
bad	8	high	high	high	low	70to74	america
bad	6	medium	medium	medium	medium	70to74	america
bad	4	low	medium	low	medium	70to74	asia
bad	4	low	medium	low	low	70to74	asia
bad	8	high	high	high	low	75to78	america
:	:	:	:	:	:	:	:
:	:	:	:	:	:	:	:
:	:	:	:	:	:	:	:
bad	8	high	high	high	low	70to74	america
good	8	high	medium	high	high	79to83	america
bad	8	high	high	high	low	75to78	america
good	4	low	low	low	low	79to83	america
bad	6	medium	medium	medium	high	75to78	america
good	4	medium	low	low	low	79to83	america
good	4	low	low	medium	high	79to83	america
bad	8	high	high	high	low	70to74	america
good	4	low	medium	low	medium	75to78	europa
bad	5	medium	medium	medium	medium	75to78	europa

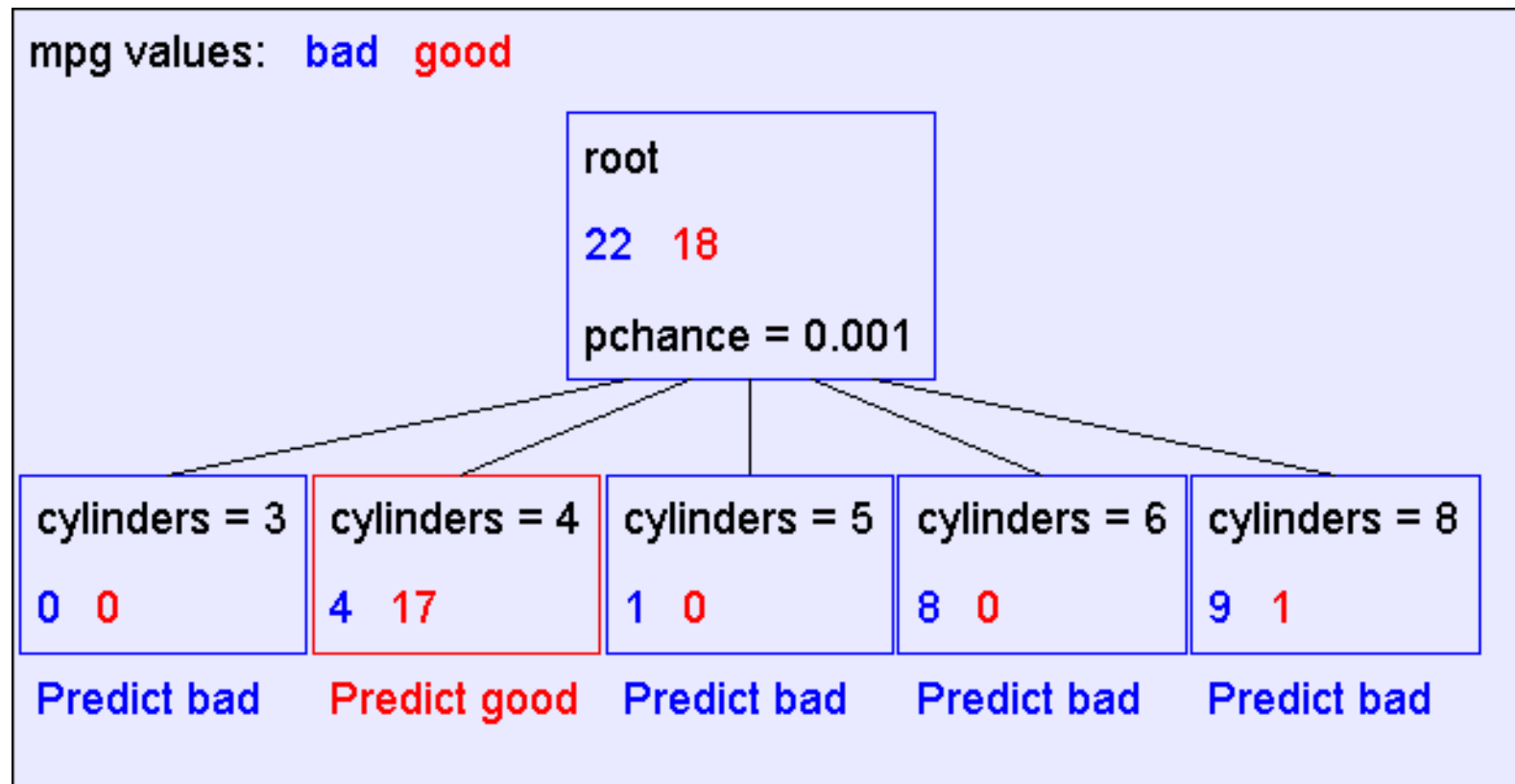
- Look at all the information gains...

Information gains using the training set (40 records)

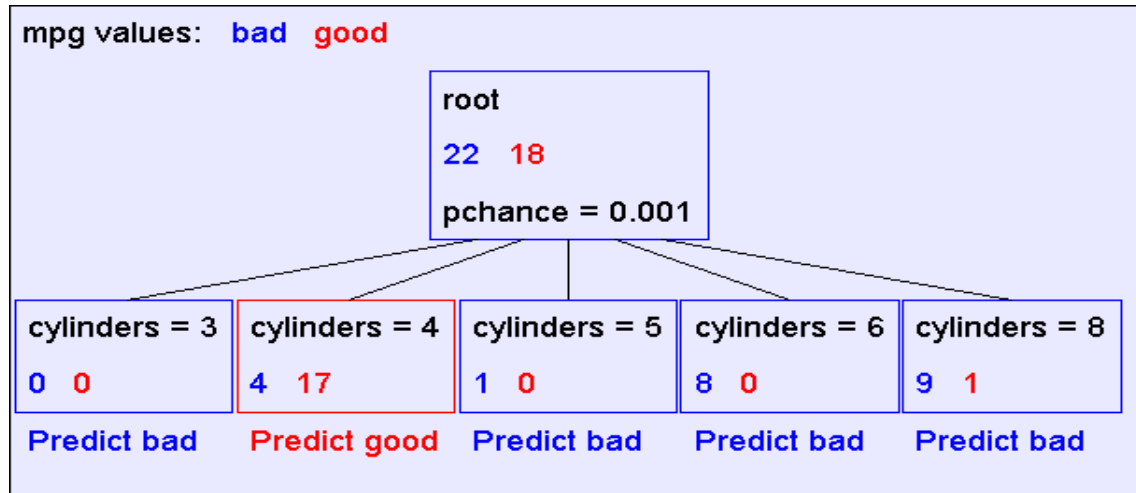
mpg values: bad good

Input	Value	Distribution	Info Gain
cylinders	3		0.506731
	4		
	5		
	6		
	8		
displacement	low		0.223144
	medium		
	high		
horsepower	low		0.387605
	medium		
	high		
weight	low		0.304018
	medium		
	high		
acceleration	low		0.0642088
	medium		
	high		
modelyear	70to74		0.267964
	75to78		
	79to83		
maker	america		0.0437265
	asia		

A Decision Stump



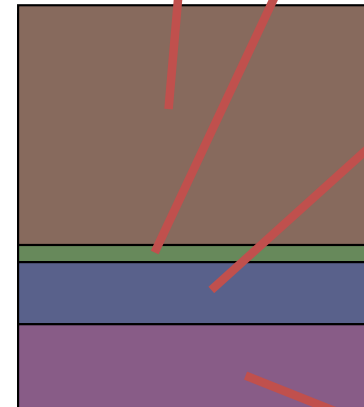
Recursion Step



Take the
Original
Dataset..



And partition it
according
to the value of
the attribute
we split on



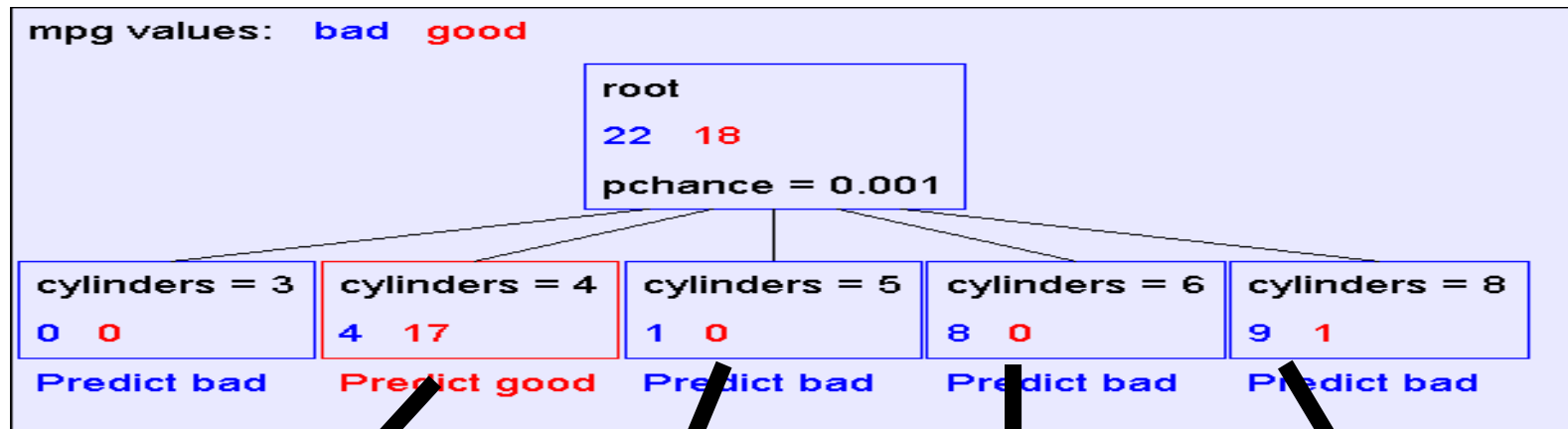
Records
in which
cylinders
= 4

Records
in which
cylinders
= 5

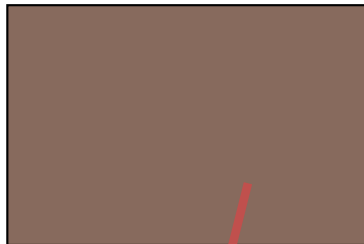
Records
in which
cylinders
= 6

Records
in which
cylinders
= 8

Recursion Step



Build tree from
These records..



Records in
which cylinders
= 4

Build tree from
These records..



Records in
which cylinders
= 5

Build tree from
These records..



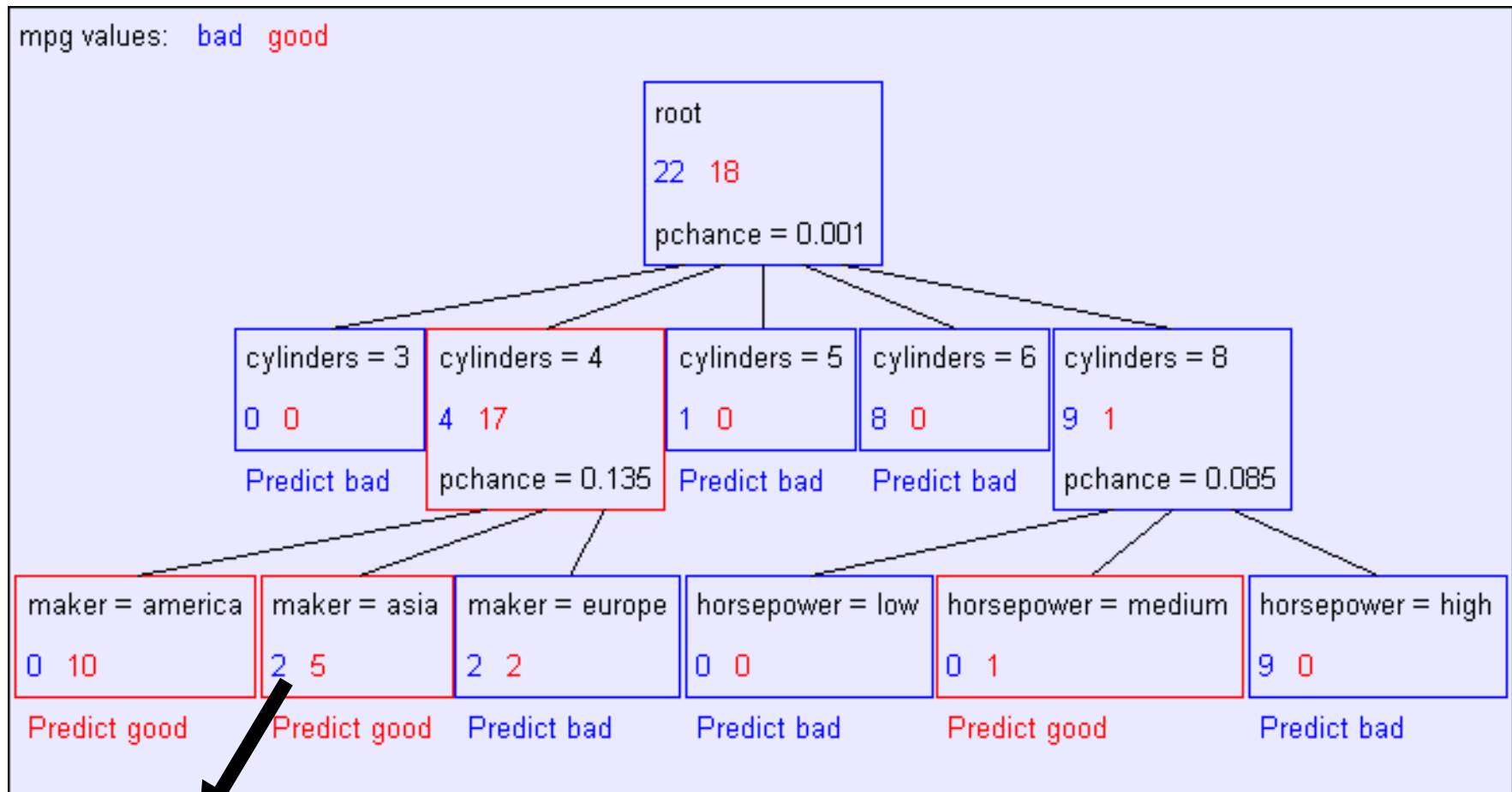
Records in
which cylinders
= 6

Build tree from
These records..



Records in
which cylinders
= 8

Second level of tree

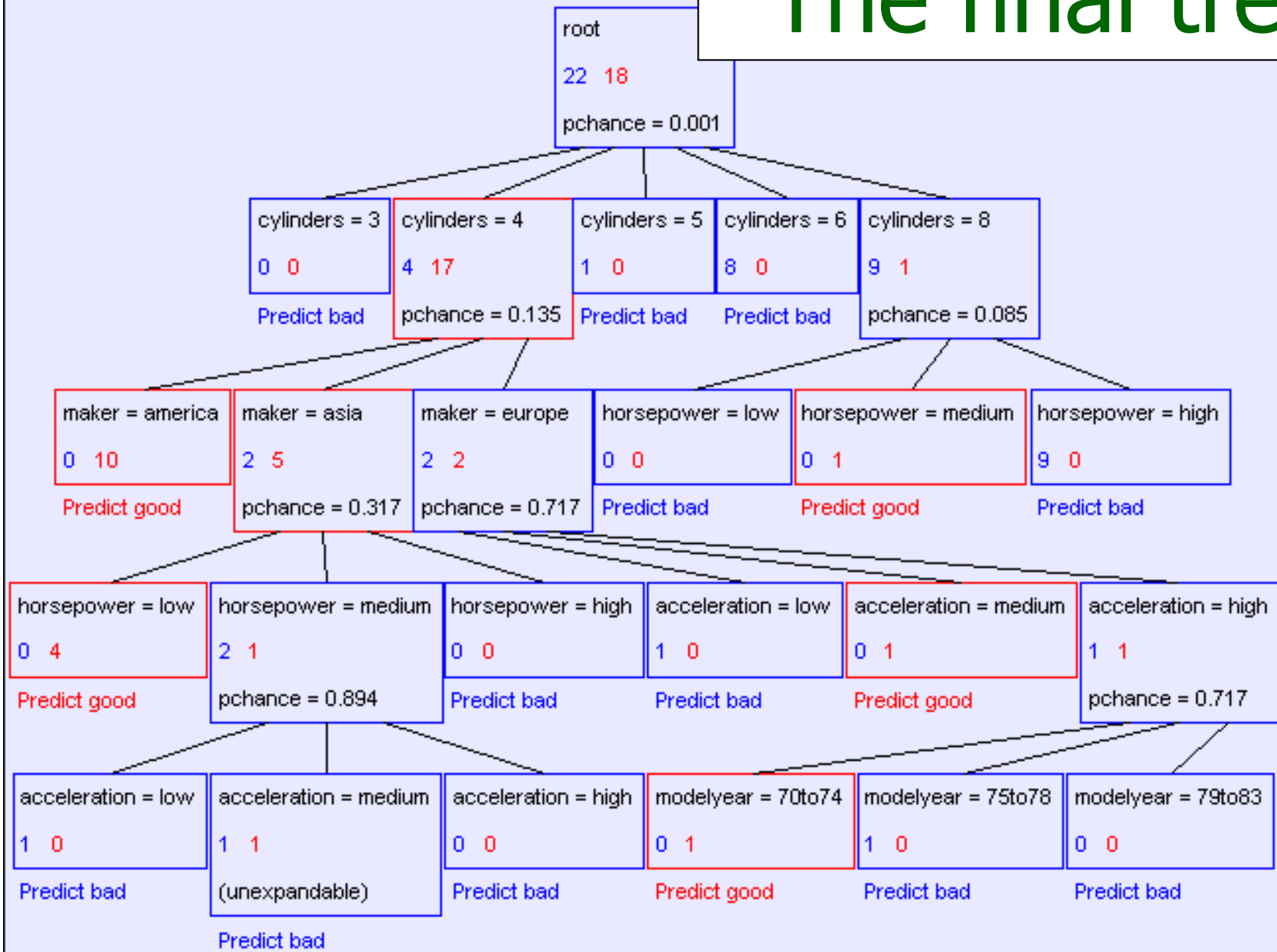


Recursively build a tree from the seven records in which there are four cylinders and the maker was based in Asia

(Similar recursion in the other cases)

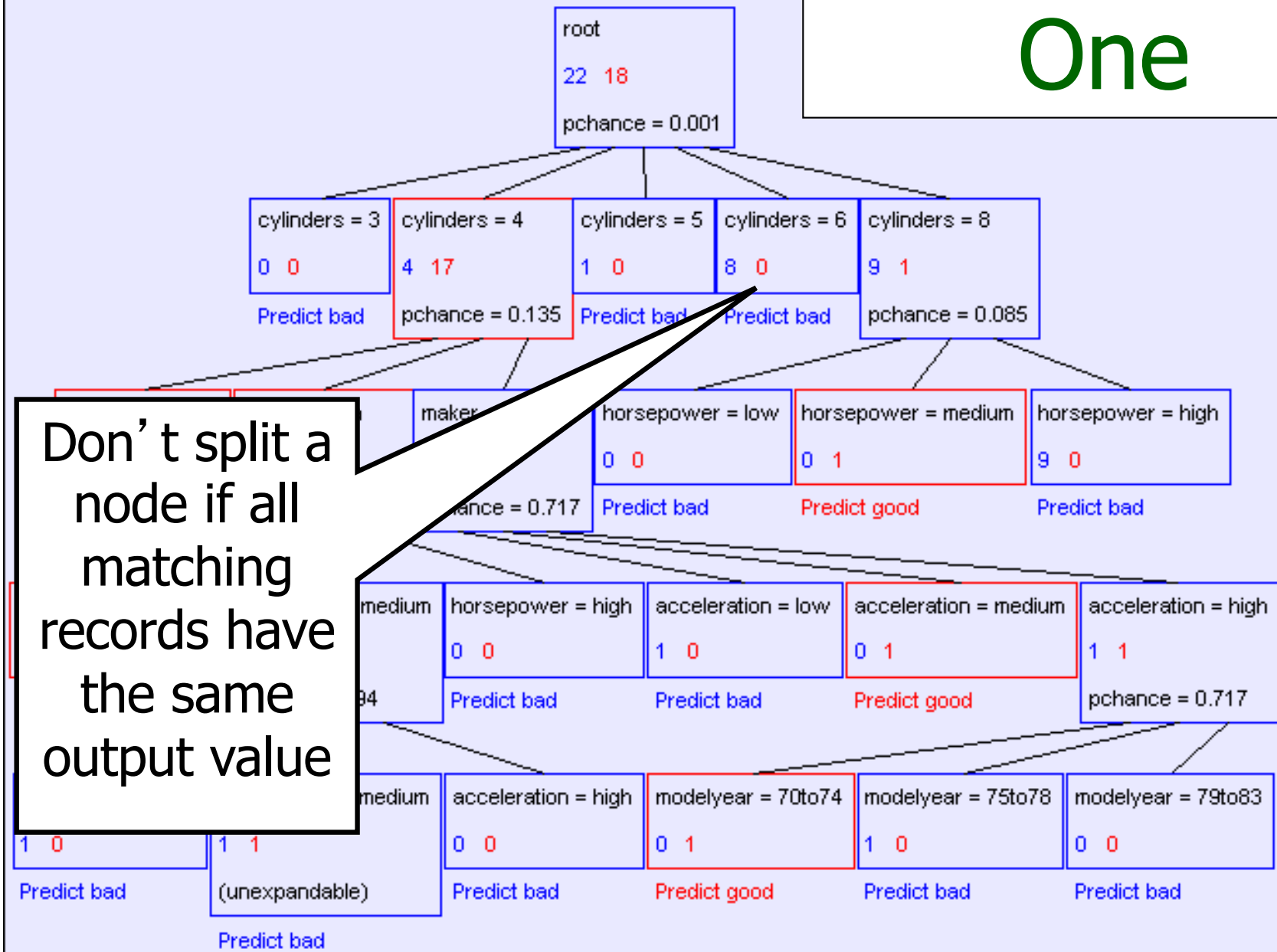
The final tree

mpg values: bad good

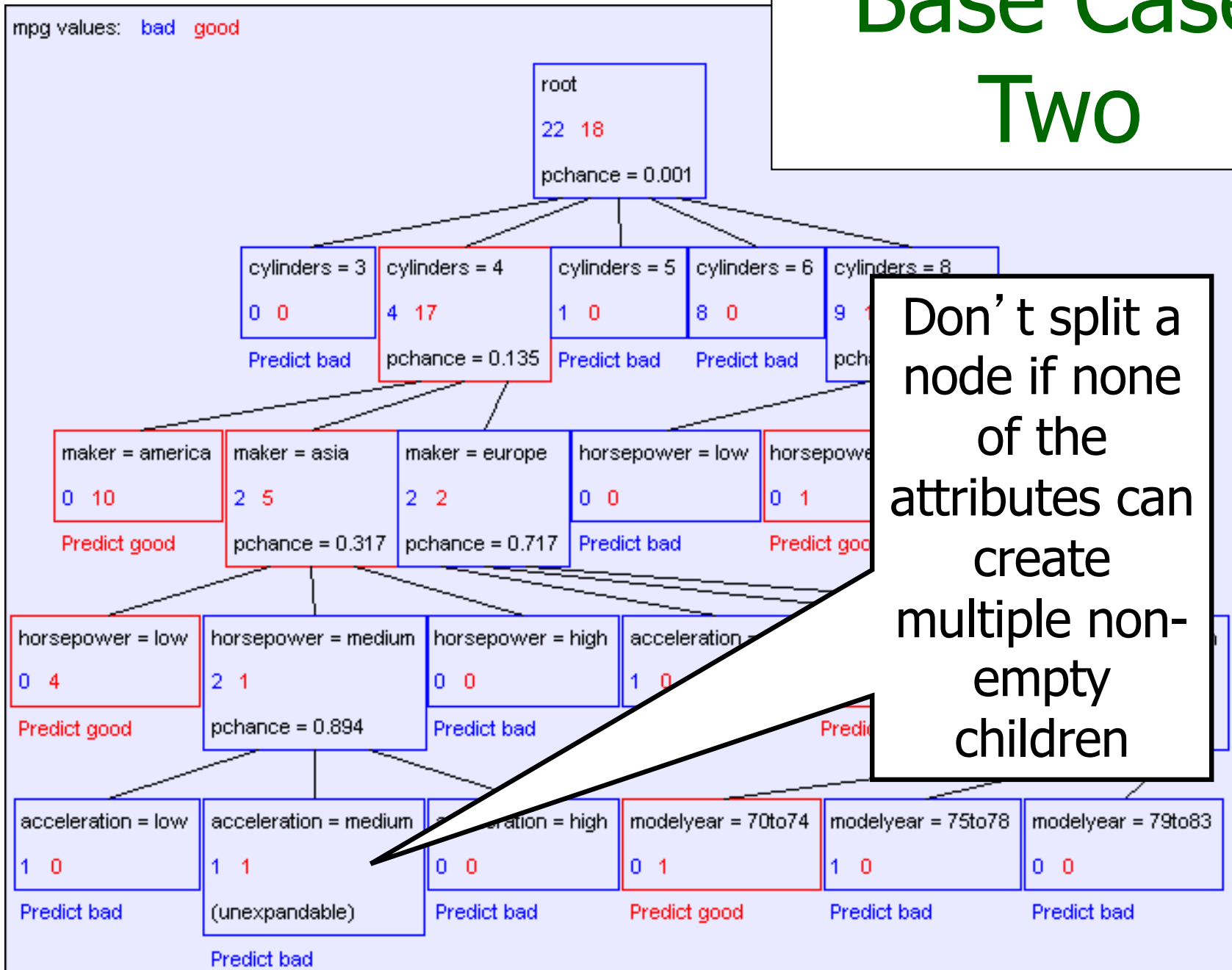


Base Case One

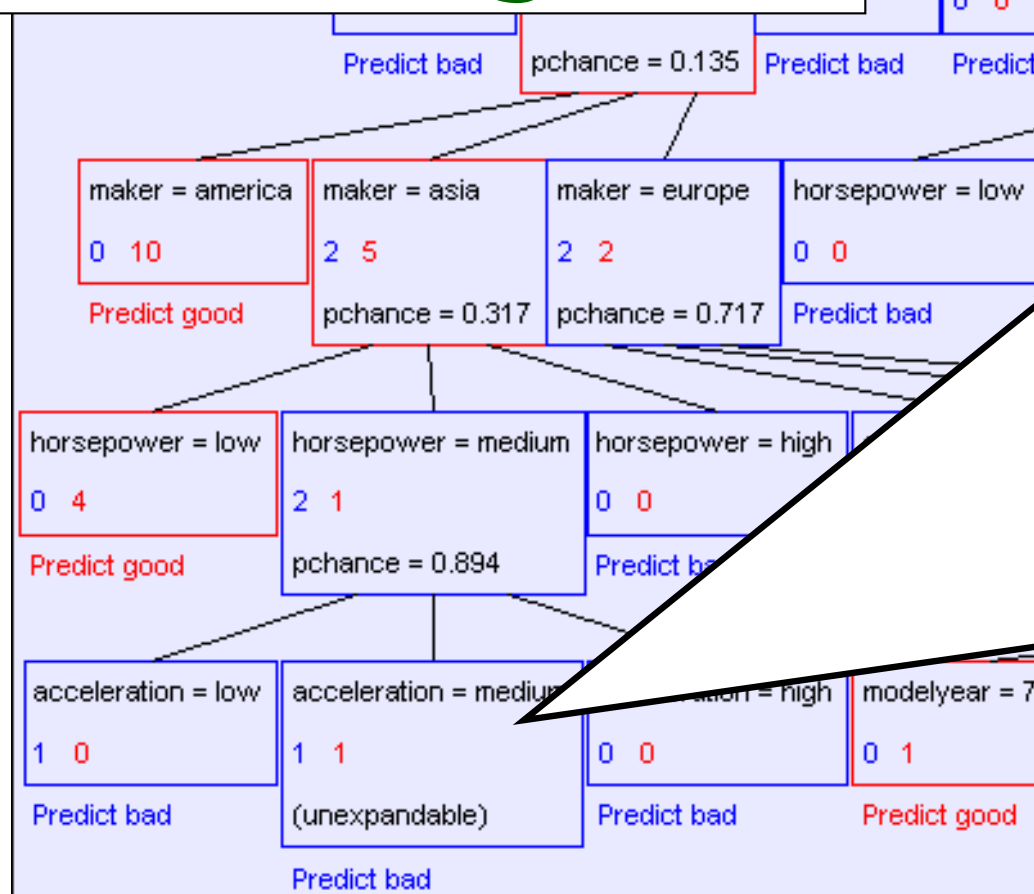
mpg values: bad good



Base Case Two



Base Case Two: No attributes can distinguish



Information gains using the training set (2 records)

mpg values: bad good

Input	Value	Distribution	Info Gain
cylinders	3		0
	4		
	5		
	6		
	8		
displacement	low		0
	medium		
	high		
horsepower	low		0
	medium		
	high		
weight	low		0
	medium		
	high		
acceleration	low		0
	medium		
	high		
modelyear	70to74		0
	75to78		
	79to83		
maker	america		0
	asia		
	europe		

Base Cases: An idea

- Base Case One: If all records in current data subset have the same output then **don't recurse**
- Base Case Two: If all records have exactly the same set of input attributes then **don't recurse**

Proposed Base Case 3:

If all attributes have zero information gain then **don't recurse**

• *Is this a good idea?*

Training Set Error

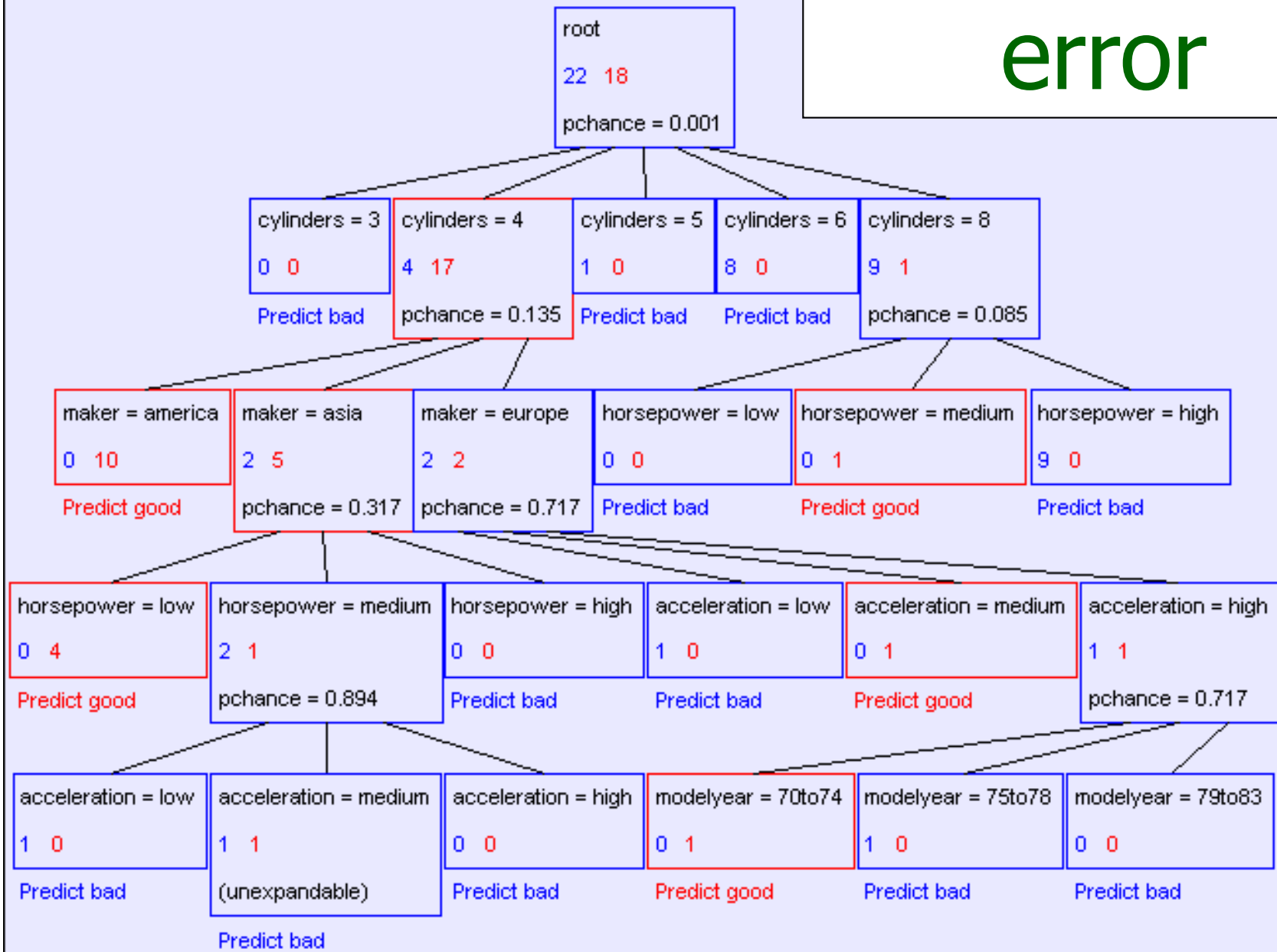
- For each record, follow the decision tree to see what it would predict

For what number of records does the decision tree's prediction disagree with the true value in the database?

- This quantity is called the *training set error*. The smaller the better.

MPG Training error

mpg values: bad good

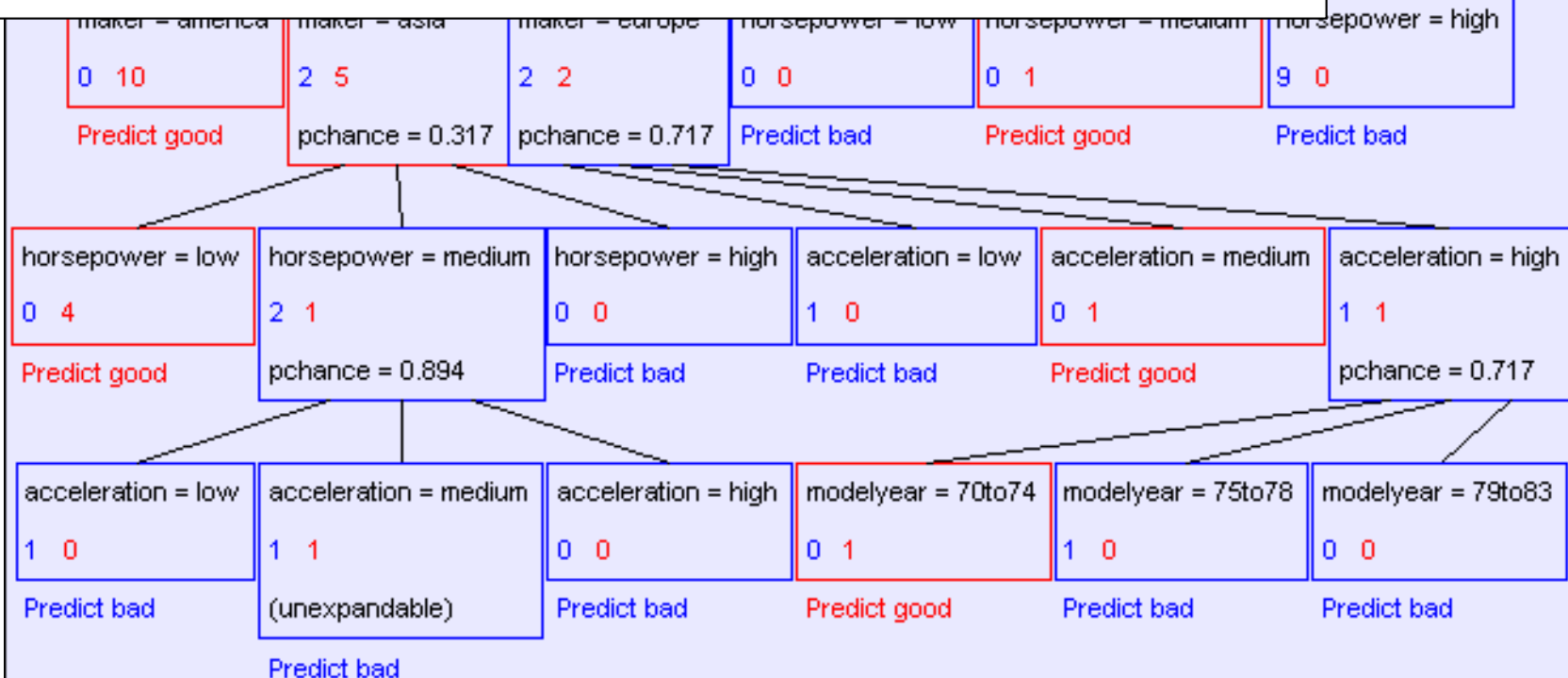


MPG Training error

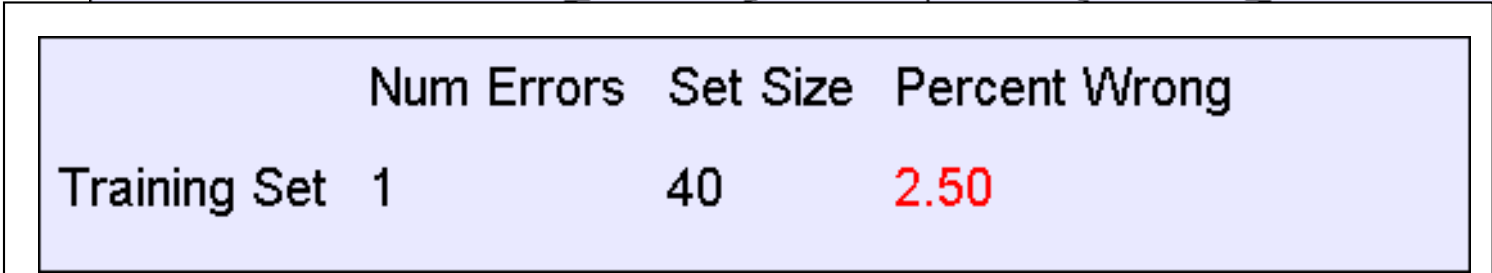
mpg values: bad good

root
22 18
pchance = 0.001

	Num Errors	Set Size	Percent Wrong
Training Set 1	1	40	2.50



MPG Training error



MPG Test set error

mpg values: bad good

root
22 18
pchance = 0.001

Num Errors Set Size Percent Wrong

Training Set	1	40	2.50
Test Set	74	352	21.02

epower = high

Predict bad

horsepower = low

0 4

Predict good

horsepower = medium

2 1

pchance = 0.894

horsepower = high

0 0

Predict bad

acceleration = low

1 0

Predict bad

acceleration = medium

0 1

Predict good

acceleration = high

1 1

pchance = 0.717

acceleration = low

1 0

Predict bad

acceleration = medium

1 1

(unexpandable)

Predict bad

acceleration = high

0 0

Predict bad

modelyear = 70to74

0 1

Predict good

modelyear = 75to78

1 0

Predict bad

modelyear = 79to83

0 0

Predict bad

MPG Test set error

mpg values: bad good

root

22 18

pchance = 0.001

	Num Errors	Set Size	Percent Wrong
Training Set	1	40	2.50
Test Set	74	352	21.02

epower = high

dict bad

horsepower = low

horsepower = medium

horsepower = high

acceleration = low

acceleration = medium

acceleration = high

0 4

0 4

0 0

4 0

0 4

4 4

Pr

ac

1

Predict bad

(unexpandable)

Predict bad

Predict good

Predict bad

Predict bad

= 0.717

= 79to83

The test set error is much worse than the training set error...

...why?

Overfitting

- Definition: If the learning algorithm fits noise (i.e. pays attention to parts of the data that are irrelevant) it is **overfitting**.
- Fact (theoretical and empirical): If the learning algorithm is overfitting, it often perform less well on test set data.

Overfitting Case Study

Consider this data:

Color	Shape	Size	Class
Red	Square	Big	+
Red	Round	Big	+
Blue	Square	Big	−
Blue	Round	Small	−
Red	Square	Small	+
Blue	Square	Small	−
Red	Round	Small	− (noise)

Most data follows:

Red → +

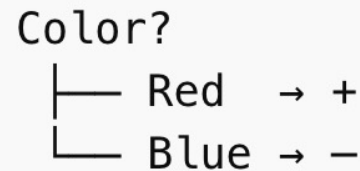
Blue → −

Except for one example: Red, Round, Small → −

Overfitting Case Study

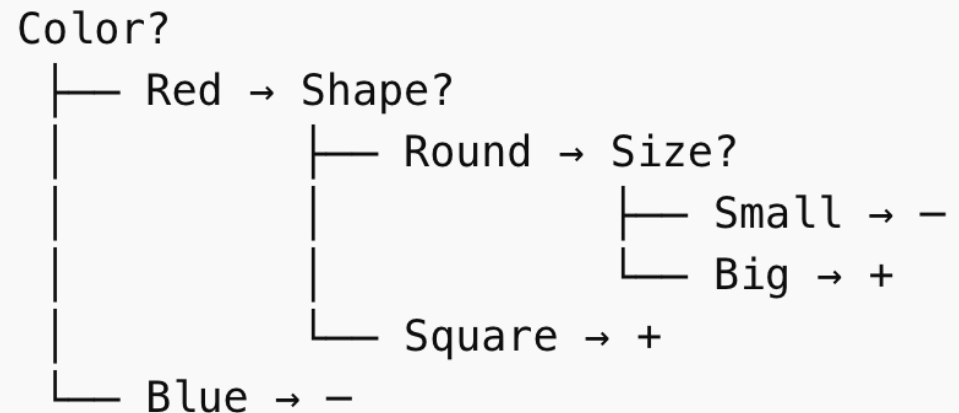
- Simple tree:

- Split on Color
- Works for most data
- One mistake
- Good generalization



- Deep tree:

- Keeps splitting (Shape, Size...)
- Eventually isolates the noisy point
 - Training error = 0
 - But model becomes **complicated + brittle**



Avoiding overfitting

- Usually we do not know in advance which are the irrelevant variables
- ...and it may depend on the context

But we can use simple statistics to warn us that we might be overfitting.

Pruning to avoid overfitting

- Pchance: p-value (statistical significance)
 - What is the probability that this split happened just by random chance?”
 - Small pChance (e.g., < 0.05) $\rightarrow$ split is **statistically meaningful**
 - Large pChance $\rightarrow$ split is likely **noise**
 - If using **Chi-square statistics**:
 - The p-value is defined as the probability of observing a value **at least as extreme as the current statistic**, assuming null hypothesis is true
 - Larger $\chi^2 \rightarrow$ more extreme $\rightarrow$ smaller p-value

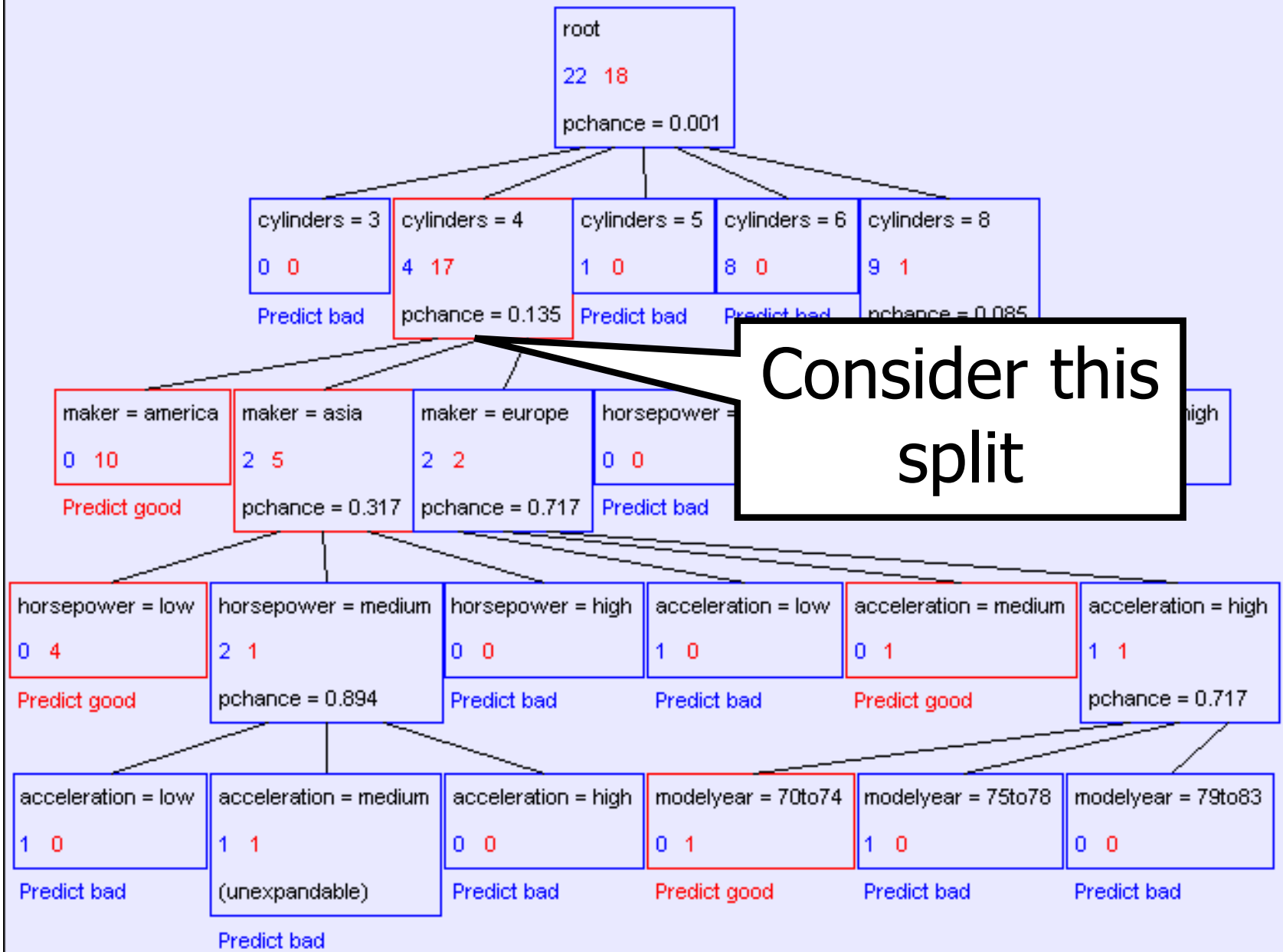
χ^2 Pruning to avoid overfitting

- Assume null hypothesis
 - Feature and class are independent (no real relationship)
- Compare **Observed vs Expected**
 - **Expected counts** = what we would see if the split were random
 - **Observed counts** = what we actually see in data
- Small difference → likely random → **large pChance** → **weak split**
- Large difference → unlikely random → **small pChance** → **strong split**

Pruning to avoid overfitting

- After building the full decision tree, start to prune:
 - Beginning at the bottom of the tree, delete splits in which $p_{chance} > MaxPchance$.
 - Continue working your way up until there are no more prunable nodes.

mpg values: bad good



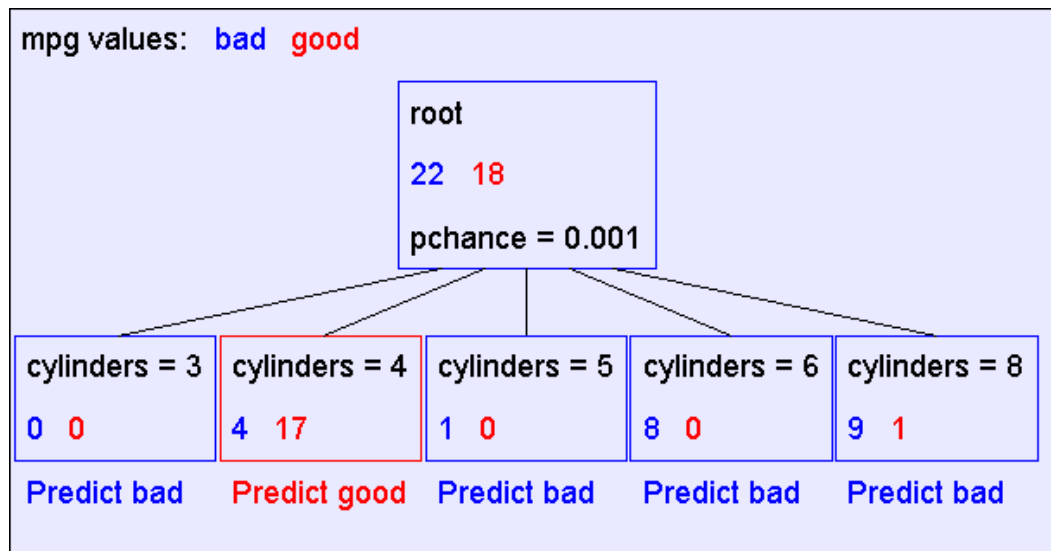
Test for significance

mpg values:		bad	good		
maker	america	0	10		 $H(\text{mpg} \mid \text{maker} = \text{america}) = 0$
	asia	2	5		 $H(\text{mpg} \mid \text{maker} = \text{asia}) = 0.863121$
	europa	2	2		 $H(\text{mpg} \mid \text{maker} = \text{europa}) = 1$
$H(\text{mpg}) = 0.702467$ $H(\text{mpg} \mid \text{maker}) = 0.478183$					
$IG(\text{mpg} \mid \text{maker}) = 0.224284$					

- Suppose that mpg was completely uncorrelated with maker.
- What is the chance we'd have seen data of at least this apparent level of association anyway?
 - Is the association really significant or by chance?

Pruning example

- With $\text{MaxPchance} = 0.1$, you will see the following MPG decision tree:



Note the improved test set accuracy compared with the unpruned tree

	Num Errors	Set Size	Percent Wrong
Training Set	5	40	12.50
Test Set	56	352	15.91

MaxPchance

- **Good news:** The decision tree can automatically adjust its pruning decisions according to the amount of apparent noise and data.
- **Bad news:** The user must come up with a good value of MaxPchance.
- **Good news:** But with extra work, the best MaxPchance value can be estimated automatically by cross-validation.

Gain Ratio

- Problem with Information Gain

- Favors many smaller partitions formed by attribute with many unique values, i.e., birthday as a feature
- Gain Ratio takes into account the number of branches generated from the split test:

$$\text{split info}(X) = - \sum_{i=1}^n \frac{|T_i|}{|T|} \times \log_2 \left(\frac{|T_i|}{|T|} \right)$$

$$\text{Gain Ratio}(X) = \frac{\text{gain}(X)}{\text{split info}(X)}$$

Attribute X with
n possible values
|T|: size of the entire data
|T_i|: size of the subset data T_i
with attribute value X_i

Case 1: 90% / 10%

$$\text{SplitInfo} = -(0.9 \log_2 0.9 + 0.1 \log_2 0.1) \approx \mathbf{0.469}$$

Case 2: 10 branches, each 10%

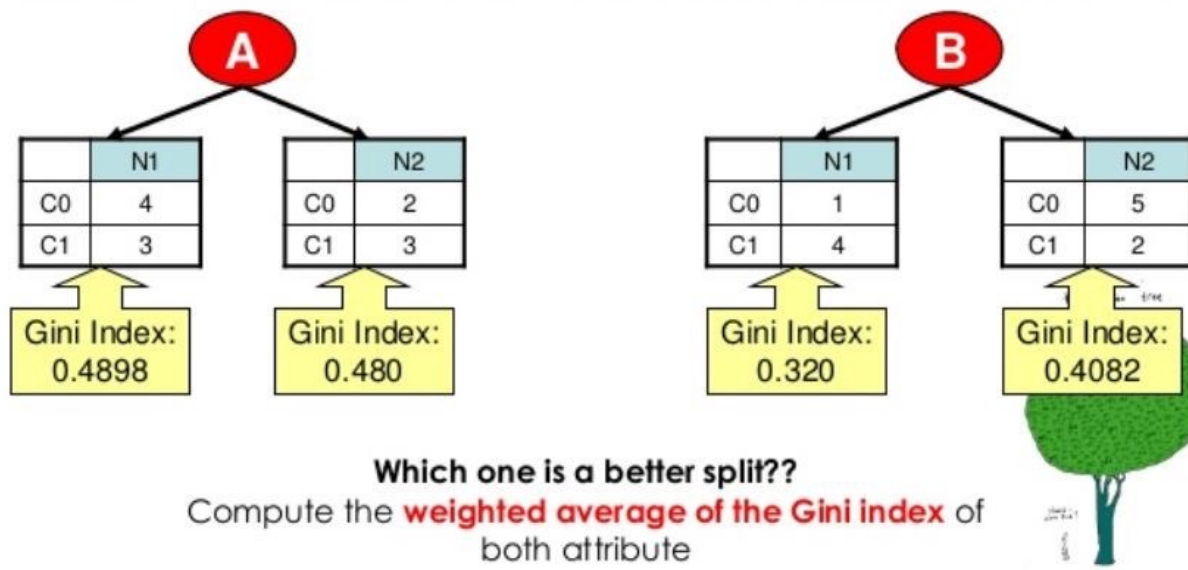
$$\text{SplitInfo} = - \sum_{i=1}^{10} 0.1 \log_2 0.1 = -10 \cdot (0.1 \log_2 0.1) \approx \mathbf{3.322}$$

Gini Index

- Another impurity criterion favoring larger partitions:

$$Gini = 1 - \sum_j p_j^2$$

Suppose there are two ways (A and B) to split the data into smaller subset.



The smaller the G value, the purer the node

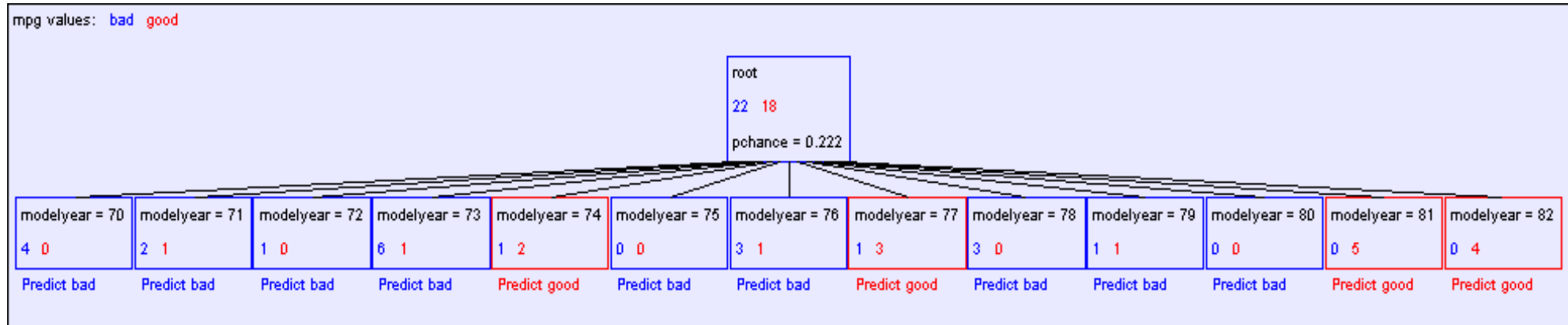
Real-Valued inputs

- What should we do if some of the inputs are real-valued?

mpg	cylinders	displacement	horsepower	weight	acceleration	modelyear	maker
good	4	97	75	2265	18.2	77	asia
bad	6	199	90	2648	15	70	america
bad	4	121	110	2600	12.8	77	europa
bad	8	350	175	4100	13	73	america
bad	6	198	95	3102	16.5	74	america
bad	4	108	94	2379	16.5	73	asia
bad	4	113	95	2228	14	71	asia
bad	8	302	139	3570	12.8	78	america
:	:	:	:	:	:	:	:
:	:	:	:	:	:	:	:
:	:	:	:	:	:	:	:
good	4	120	79	2625	18.6	82	america
bad	8	455	225	4425	10	70	america
good	4	107	86	2464	15.5	76	europa
bad	5	131	103	2830	15.9	78	europa

Idea One: Branch on each possible real value

One branch for each numeric value



Hopeless: with such high branching factor will shatter the dataset and over fit

Note pchance is 0.222 in the above...if MaxPchance was 0.05 that would end up pruning away to a single root node.

A better idea: thresholded splits

- Suppose X is real valued.
- Define $IG(Y/X:t)$ as $H(Y) - H(Y/X:t)$
- Define $H(Y/X:t) =$
$$H(Y/X < t) P(X < t) + H(Y/X \geq t) P(X \geq t)$$
 - $IG(Y/X:t)$ is the information gain for predicting Y if all you know is whether X is greater than or less than t
- Then define $IG^*(Y/X) = \max_t IG(Y/X:t)$
- For each real-valued attribute, use $IG^*(Y/X)$ for assessing its suitability as a split

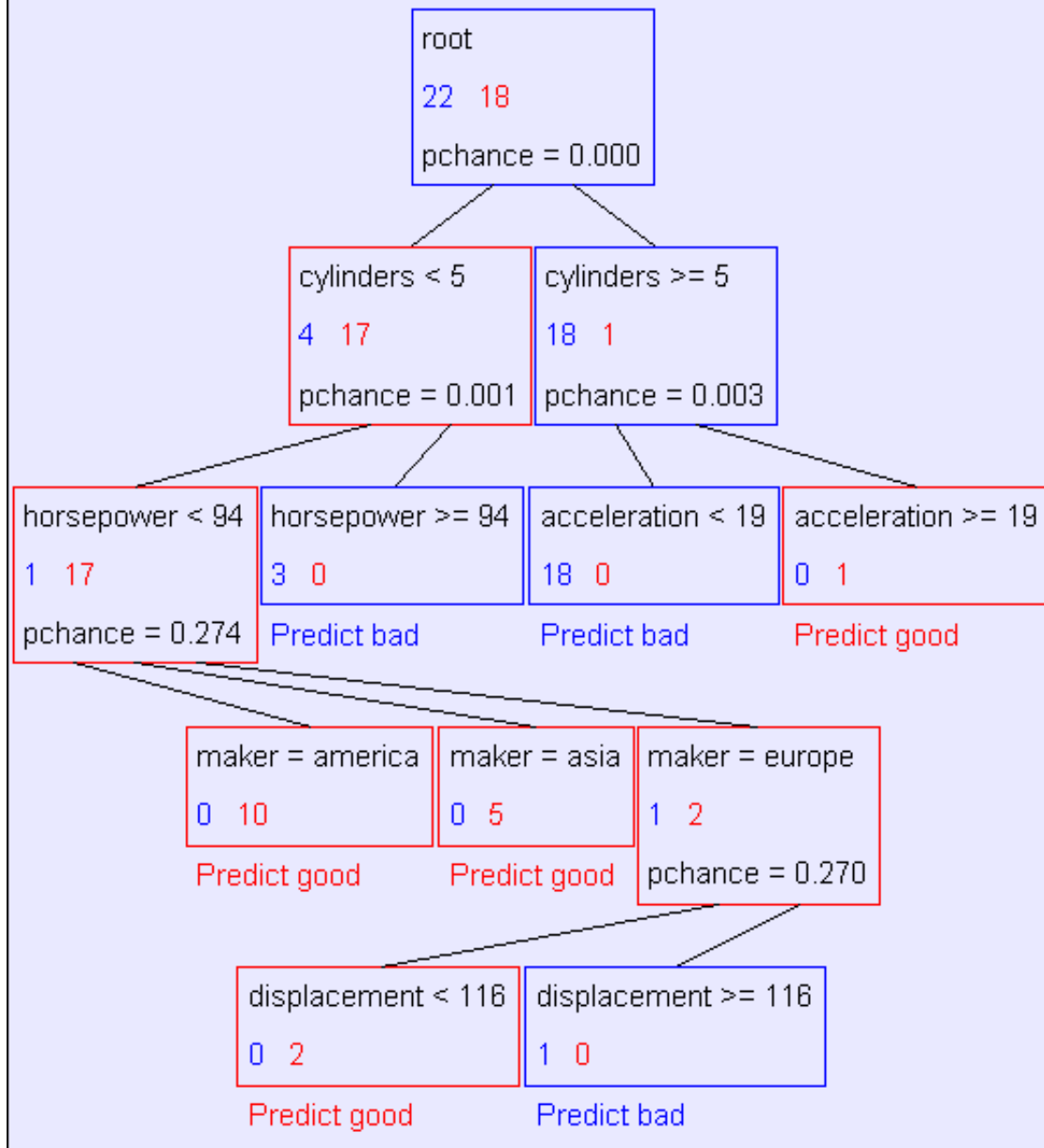
- Example with MPG

Information gains using the training set (40 records)

mpg values: bad good

Input	Value	Distribution	Info Gain
cylinders	< 5		0.48268
	>= 5		
displacement	< 198		0.428205
	>= 198		
horsepower	< 94		0.48268
	>= 94		
weight	< 2789		0.379471
	>= 2789		
acceleration	< 18.2		0.159982
	>= 18.2		
modelyear	< 81		0.319193
	>= 81		
maker	america		0.0437265
	asia		
	europa		

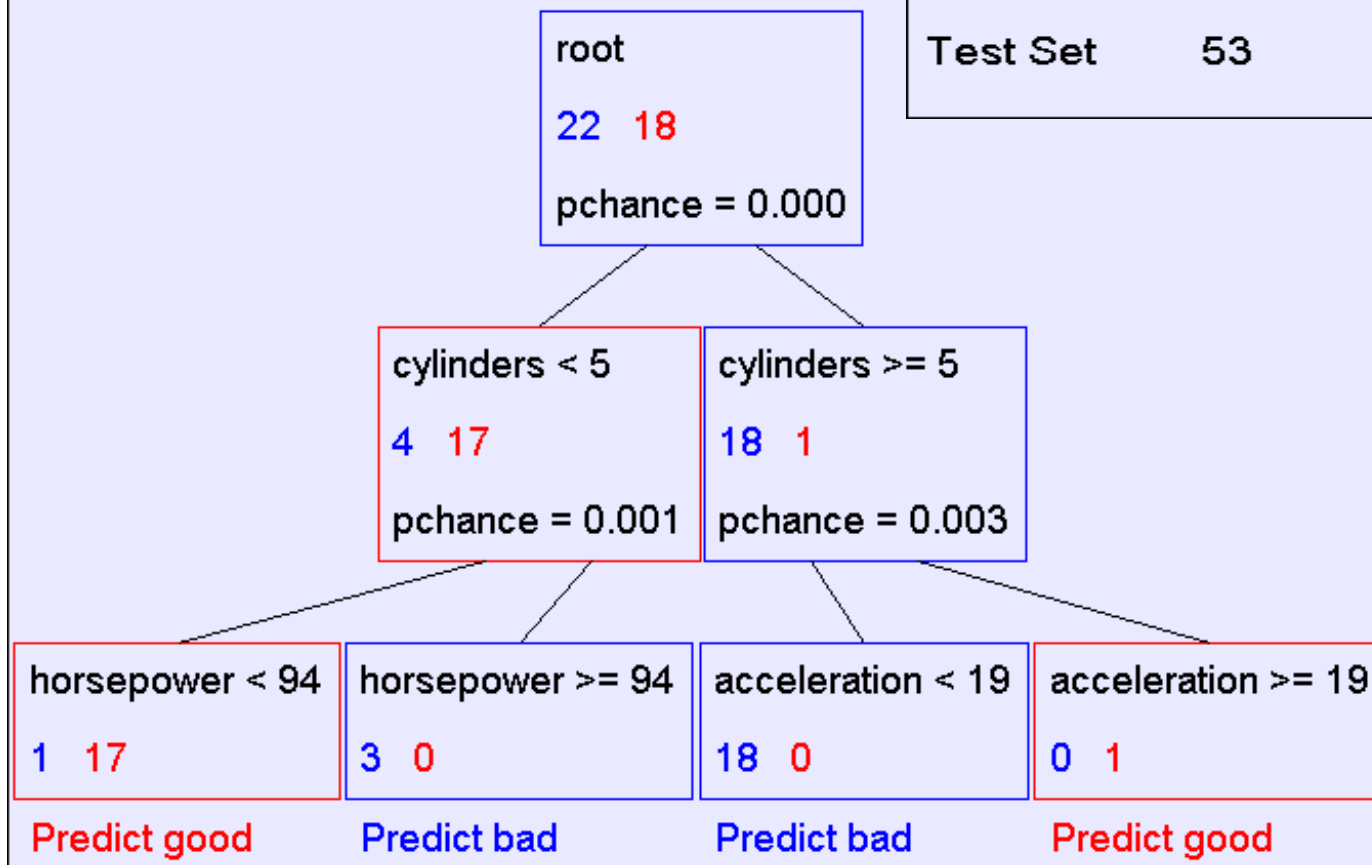
mpg values: bad good



Unpruned
tree using
reals

Pruned tree using reals

mpg values: **bad** **good**



		Num Errors	Set Size	Percent Wrong
Training Set	1	40		2.50
Test Set	53	352		15.06

Set MaxChance = 0.003

Additional Considerations

- How to deal with Missing Data?
- Other tree pruning methods
 - Cost-complexity pruning
 - Reduced-error pruning

In Summary

- Decision trees has been a popular data mining tool
 - Easy to understand
 - Easy to implement
 - Easy to use
 - Computationally cheap
- It's possible to get in trouble with overfitting
- They do classification: predict a categorical output from categorical and/or real inputs